Implementing Assignment 1 by Adding Word Embedding and LSTM Layers Before the Hidden Layers of the Neural Network on the English IMDB Corpus and Two Persian Corpora

A) In this section, we are required to use an Embedding layer with three different dimensions selected from the following values: 8, 10, 16, 32, 64, and 100, with input_length=50.
The Embedding layer converts text into numerical vectors, assigning a numerical representation to each word. The different dimensions specified in the assignment represent the length of the numerical vector used to represent each word.
In the code, we create an array containing the specified values and train the model separately using each dimension. This allows us to identify the embedding dimension that achieves the highest accuracy among the tested values.

Results for the IMDB dataset:

F1Score	Recall	Precision	Accuracy	Embedding Dimension
0.82269	0.83112	0.81444	0.82088	8
0.82157	0.82808	0.8151	0.82016	10
0.81485	0.80591	0.8239	0.81688	16
0.8146	0.82408	0.8054	0.81252	32
0.81246	0.82088	0.8042	0.81052	64
0.81106	0.8208	0.80156	0.8088	100

We observe that the highest accuracy is achieved with embedding dimensions of 8 and 10, after which the accuracy decreases. This may be related to the number of words in the dataset. Beyond a certain dimension (10 in this case), the word vectors may become unnecessarily large without providing additional benefits. Considering the model configuration and the vocabulary size of 10,000 defined for this dataset, an embedding dimension of 10 appears to be the most suitable choice.
Based on the table and the code executed in Google Colab, we can also observe that the model performs better on the training data than on the validation data, achieving higher accuracy during training. This indicates that the model may be overfitting.
Furthermore, as mentioned earlier, the model achieves better accuracy with smaller embedding dimensions. The F1-score, Precision, and Recall values are also higher in these configurations.

A higher Precision indicates fewer False Positives, meaning that when the model predicts a sample as Positive, it is more likely to be correct.
A higher Recall indicates fewer False Negatives, meaning that the model is better at identifying actual Positive samples.
The F1-score is a value between 0 and 1, with higher values indicating better overall classification performance.

Results for the second dataset:

F1Score	Recall	Precision	Accuracy	Embedding Dimension
0.8491	0.8286	0.8727	0.8531	8
0.8459	0.8060	0.8899	0.8531	10
0.8491	0.8309	0.8681	0.8524	16
0.8389	0.8003	0.8815	0.8446	32
0.8353	0.8186	0.8526	0.8385	64
0.8320	0.8040	0.8621	0.8377	100

For this dataset, we also observe that smaller embedding dimensions achieve better performance, approximately following the same pattern discussed for the first dataset.

F1Score	Recall	Precision	Accuracy	Embedding Dimension
0.8414	0.86	0.8631	0.8600	8
0.8854	0.89	0.8971	0.8899	10
0.9309	0.935	0.9396	0.9350	16
0.9640	0.965	0.9671	0.9649	64
0.9694	0.97	0.9717	0.9700	100

In contrast, for the third dataset, we observe that the model performs better as the embedding dimension increases. Ultimately, the best performance is achieved with an embedding dimension of 100.

This may be related to the characteristics of the dataset. In this case, representing the words in the dataset using 100-dimensional vectors may provide a more effective representation, resulting in higher model accuracy.

The code for this section is available in File A.

B) Using Pretrained Embedding Models

In Part B, we are required to use pretrained embedding models. In my implementation, I used GloVe.

According to the uploaded results, we observe that the accuracy on the IMDB dataset decreases compared to Part A. Therefore, using the pretrained GloVe model did not help improve our model's accuracy.

In general, Embedding is task-specific. When we have sufficient training data, adding an Embedding layer ourselves allows the network to learn word representations tailored to the specific task and the words in that dataset. As a result, the model may perform better. However, as stated in the textbook, pretrained models can be useful when we do not have enough data for training.

In this case, we have sufficient training data, which explains why the model in Part A, where we added and trained the Embedding layer ourselves, performs better.

For the Persian datasets, we also observe a decrease in accuracy compared to Part A. One reason is that the pretrained GloVe model does not contain certain words, such as Persian words related to food or quality. Consequently, no word embeddings are available for these words, meaning that they do not have corresponding numerical vector representations in the GloVe model. Using the pretrained GloVe model therefore reduces accuracy compared to Part A. This pretrained model does not perform well on these Persian datasets, and training our own Embedding layer would be a better approach.

The same explanations discussed for the IMDB dataset also apply here. For the third dataset, the accuracy decreases significantly as well.

The code for this section is available in File B.

C) Adding SimpleRNN Layers

In Part C, we are required to add two SimpleRNN layers to the best-performing model from Part A.

We observe that, for the first dataset (IMDB), adding two SimpleRNN layers improves the model's accuracy and overall performance.

The evaluation metrics are also presented below.

```
Confusion Matrix:
[[9688 2812]
 [2942 9558]]
Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.78      0.77     12500
```

1	0.77	0.76	0.77	12500

We can conclude that adding RNN layers improves the model's performance. This is related to the way RNNs process sequential data. Words are fed into the RNN one by one, allowing the network to process them sequentially. Through its feedback loop, the RNN updates its weights as it processes the inputs and preserves information about the order of the words.
By taking word order into account, the network can better capture semantic and syntactic relationships between words. As a result, the neural network can understand the meaning of sentences more effectively, leading to improved accuracy.

For the other two datasets (the Persian datasets), the accuracy is reasonably good and higher than that achieved in Part B. However, it does not exceed the best accuracy obtained in Part A.
The evaluation metrics and Confusion Matrix for each dataset are included in the code. Since the meanings of these evaluation metrics were already explained in this assignment and the previous one, I will avoid repeating those explanations and focus instead on analyzing the final results.
For both Persian datasets, however, Precision and Recall, like the model's accuracy, do not reach the levels achieved by the best-performing model in Part A.

D) Adding LSTM Layers

In this section, we are required to add two LSTM layers to the models from Part B.

The models in Part B used the pretrained GloVe model for their Embedding layers and achieved lower accuracy than the models in Part A.

After adding two LSTM layers, we observe that the accuracy on the IMDB dataset increases slightly compared to Part B. This improvement is related to the characteristics of LSTM, which can process sequences of inputs and handle textual data more effectively.

For the Persian datasets, adding LSTM layers also results in a slight improvement in accuracy. As mentioned earlier, this is due to the characteristics of LSTM, particularly its long-term memory and short-term memory capabilities, as well as the way it processes sequential inputs. These features contribute to a modest improvement in the models' accuracy.

The code for this section, along with the evaluation metrics, is available in File D.